Distributed Sensor Hub

Final Report for the Distributed Systems Course 2025/2026

Alex Santini
alex.santini2@studio.unibo.it

May 1, 2026

Distributed Sensor Hub is a fully decentralized peer-to-peer platform for smart-city sensor data collection and replication, designed to operate across multiple nodes without any central coordinator. Each node may manage connections to multiple local sensors, ingest their readings through a pluggable sensor interface to support different sensor types, and maintain a local replica of the shared system state.

The platform disseminates sensor updates through push-pull gossip and resolves conflicts with deterministic timestamp-based Last-Writer-Wins merging, providing eventual consistency across replicas.

To remain operational in realistic distributed environments, the system combines gossip-based membership, phi-accrual failure detection, and anti-entropy recovery after crashes or temporary network partitions. In CAP terms, the project prioritizes availability and partition tolerance over strong consistency, while still guaranteeing convergence once communication is restored.

Disclaimer

During the development of the project and the preparation of this report, the author used Large Language Models (LLMs) as auxiliary tools for limited coding support, brainstorming, and improving the clarity and structure of the written text.

All outputs produced with the assistance of LLMs were critically reviewed, validated, and, when necessary, revised by the author. All architectural decisions, system design choices, and implementation details were defined and verified by the author, who takes full responsibility for the final content of this report.

1 Concept

Product type. *Distributed Sensor Hub* is a decentralized peer-to-peer software platform for distributed sensor monitoring. It combines:

- a set of autonomous, containerizable peer nodes that collect local readings, synchronize state in a distributed cluster, and replicate data without a central coordinator;
- an HTTP API for observability and integration;
- a read-only web UI for monitoring.

Use case description. What is the software doing? Each node maintains active connections to its local sensors, continuously collects readings, and updates its own local state. In parallel, the node shares its sensor-state updates with peer nodes and receives their updates through a gossip-based push-pull protocol. As a result, every node progressively learns and replicates the state produced by other nodes, building a distributed shared view. Eventual consistency is guaranteed through timestamp-based conflict resolution, while the system remains operational under failures, delays, and temporary partitions.

Where are the users? In a smart-city setting, users are in two main groups. Technical operators/developers manage container deployment and node setup, and they extend the system because sensor protocols are injectable and node-to-node connection policies are pluggable; they may also build tools that consume exported data. Information-oriented users access the resulting data views to monitor conditions and support practical decisions or downstream services.

When and how frequently do they interact with the system? Interaction is role-dependent. Technical operators/developers interact mainly during setup, topology changes, protocol/policy development, testing, and incident handling; this is recurring but not continuous. Information-oriented users interact when they need situational updates, monitoring views, or derived outputs for operational decisions. In contrast, the distributed core operates continuously and autonomously: sensors generate readings periodically and nodes exchange synchronization updates continuously.

How do they interact with the system? Technical operators/developers interact through deployment/runtime tooling (local process management or Docker orchestration), protocol/policy implementation components, and programmatic inspection via HTTP endpoints. Information-oriented users interact through read-only monitoring views, typically via the web UI or external dashboards/tools that consume exported data.

Does the system need to store user data? Which? Where? No user-specific data is stored. The system handles technical runtime data only: timestamped sensor readings, replicated node state, synchronization metadata, and membership/peer information. Data is maintained locally by each node (in-memory in the current scope) and propagated across peers by the gossip protocol.

Multiple roles The model includes three technical roles:

- *Node (core role)*: ingests local sensor data, maintains local state, and participates in peer synchronization and replication;
- *Technical operator/developer*: deploys and configures nodes, develops injectable sensor protocols and pluggable connection policies, and validates runtime behavior;
- *Information-oriented user/client*: consumes read-only state views for monitoring and downstream operational use, without participating in replication.

Why distribution is needed. Distribution is a core requirement, not an implementation detail. In a smart-city environment, sensing points are geographically distributed across different physical areas, so data is naturally produced at multiple nodes rather than at a single central site. Each node contributes local observations and shares them with peers, allowing the system to build a replicated global view without relying on a permanent coordinator.

Distribution is also required for fault tolerance. Nodes share sensor state, membership knowledge, topology information, and replication metadata through peer-to-peer communication, while remaining able to operate autonomously if other peers become unreachable. This allows the system to continue local ingestion and observation during node crashes, delays, or temporary network partitions, and to converge again when communication is restored.

Finally, distribution is essential because the project is meant to expose real distributed-systems behavior, including delay, partial failure, recovery, and eventual consistency. These aspects are part of the project goals and must be observable rather than hidden behind a centralized architecture.

The detailed behavioral requirements, protocol constraints, and acceptance criteria are specified in section 2.

2 Requirements Elicitation and Analysis

This section provides functional, non-functional, and implementation requirements for the analysed system, focusing on distributed sensor ingestion, replicated-state convergence, fault observability, and reproducible deployment.

2.1 Relevant Distributed System Features

Not all distributed-system features have the same priority in this project. The primary ones are decentralization, eventual convergence, partition/failure tolerance, and observability. Production-grade security and very-large-scale elasticity are outside the current scope.

Distribution transparency. Partially relevant. Low-level details should be abstracted for clients, but distributed behavior must remain visible. Delays, membership evolution, liveness changes, and recovery are intentionally observable via API and dashboard.

Decentralization. Highly relevant. Each node runs the full runtime stack independently. Normal operation must not depend on a permanent master, central broker, or external shared database.

Fault tolerance and recoverability. Highly relevant. Surviving nodes must continue local operation during crashes or partitions, and reconnected nodes must recover missing state from reachable peers (delta sync when possible, fuller transfer when needed). Recovery is eventual and parameter-dependent, not real-time guaranteed.

Failure detection and fault observability. Highly relevant. Liveness is inferred from heartbeat evidence and exposed as states such as `alive`, `suspected`, and `dead`, together with events and replication metrics.

Consistency. Highly relevant. The model is eventual consistency with deterministic conflict resolution: replicas may diverge temporarily but converge after exchanging the same updates. Strong immediate consistency is not a goal.

Partition tolerance. Relevant. During temporary partitions, nodes continue local ingestion/observation while replicas may diverge; after healing, synchronization reconciles reachable replicas.

Scalability. Relevant at intended scale. With full-mesh connectivity, the system targets small/medium clusters; alternative injectable topology policies can support larger deployments.

Resource sharing. Highly relevant. Nodes share distributed knowledge (sensor winners, membership/liveness data, topology metadata, events, metrics) through message exchange and deterministic merge rules, not through a central store.

Openness, interoperability, and heterogeneity. Relevant. The architecture separates sensor providers, topology policies, runtime/protocol logic, replicated state, and observation interfaces, so sensor protocols and connection policies remain replaceable.

Interoperability is provided through shared read-only HTTP surfaces for dashboards, scripts, and tools.

Evolvability and maintainability. Highly relevant. The implementation should remain modular (runtime, networking, protocol, membership, replication, state, sensors, topology, API/UI) to support debugging, validation, and extension.

Performance, concurrency, and communication efficiency. Relevant, without hard real-time targets. Nodes must support concurrent sensing, membership, replication, and API serving; timing and sync parameters should stay configurable for stable behavior. No strict latency/throughput SLA is required.

Security and trust. Limited in current scope. The system assumes trusted environments and does not currently provide authentication, authorization, or transport encryption. Security hardening is a future extension.

Economy and cost. Relevant as a constraint. The project should run on commodity machines using repository artifacts and Docker Compose, without paid managed infrastructure. Cost efficiency is secondary to dependable distributed behavior.

2.2 Functional Requirements

FR01 – Multi-node autonomous operation. The system shall allow multiple nodes to start independently and participate in the same distributed deployment without requiring a central coordinator.

Acceptance criterion. When a cluster of at least two configured nodes is started, each node becomes operational as an autonomous process and exposes its local service interfaces without depending on a single master node.

FR02 – Local sensor ingestion. Each node shall be able to manage one or more local sensor sources or sensor connections and ingest their readings into the distributed system state.

Acceptance criterion. Given at least one configured sensor on a node, the node eventually exposes fresh sensor records associated with its own origin through the read API or introspection API.

FR03 – Cluster-wide sensor-state dissemination and synchronization. The system shall propagate sensor updates among nodes through decentralized peer-to-peer replication with bidirectional synchronization so that readings produced on one node contribute to a shared replicated view and become visible on other reachable nodes.

Acceptance criterion. If a node generates new sensor readings while communication paths to other nodes are available, those readings eventually appear in the replicated sensor-state view exposed by the other reachable nodes. Reachable peers exchange updates in both directions through push/pull synchronization, so each node can advertise local changes and retrieve missing records.

FR04 – Deterministic timestamp-based conflict resolution. The system shall resolve concurrent or conflicting updates through a deterministic Last-Write-Wins merge

policy so that replicas can converge to the same winning value for each logical sensor record.

Acceptance criterion. When two or more updates compete for the same logical sensor record, all nodes that receive the same set of updates eventually expose the same winner for that record.

FR05 – Decentralized membership and liveness observation. The system shall allow nodes to discover peers through bootstrap and peer-to-peer membership exchange, maintain a distributed view of cluster membership, and provide operators with a node-local assessment of peer liveness.

Acceptance criterion. After startup and sufficient message exchange, a node exposes a membership view containing the peers it has discovered directly or indirectly; if liveness evidence from a known peer degrades or stops for long enough, at least one observing node eventually reports that peer as `suspected` or `dead` in its membership/introspection output.

FR06 – Recovery and rejoin after failures or partitions. The system shall allow a node that restarts, reconnects, or rejoins after temporary unavailability to recover the replicated state needed to resume normal operation by synchronizing with reachable peers, including cases where incremental synchronization alone is insufficient.

Acceptance criterion. After a node crash, temporary isolation, or long partition, once communication with at least one sufficiently up-to-date peer is restored, the recovering node eventually exposes a replicated state aligned with the rest of the reachable cluster, using a more complete recovery path when incremental synchronization is not enough.

FR07 – Read-only observability interface. The system shall provide a read-only programmatic interface for observing sensor state, membership, topology-related information, events, and operational metrics.

Acceptance criterion. A client can query documented HTTP `GET` endpoints and obtain structured snapshots of the node's current distributed-system view without modifying runtime state.

FR08 – Human-oriented monitoring support. The system shall support human observation of the running cluster through a dashboard or equivalent visual monitoring interface.

Acceptance criterion. An observer can inspect live cluster information from a web browser through a dashboard that consumes the exposed read-only API of a node.

2.3 Non-Functional Requirements

NFR1 – Decentralization. Normal operation shall not depend on any central coordinator or single control component.

Acceptance criterion. The system continues to ingest local readings, maintain local state, and serve read-only observability data on surviving nodes even if one arbitrary peer becomes unavailable.

NFR2 – Eventual consistency. The replicated sensor state shall provide eventual, not immediate, consistency across nodes.

Acceptance criterion. In the absence of new conflicting updates and after communication is restored, reachable replicas converge to the same winning values for the same sensor records, yielding a consistent shared view across nodes.

NFR3 – Partition tolerance with degraded semantics. The system shall remain available for local operation during temporary network partitions, accepting that different partitions may temporarily diverge.

Acceptance criterion. During a partition, nodes in each connected component continue local ingestion and observation; after the partition heals, their replicas eventually converge again.

NFR4 – Fault observability. The system shall expose failures, liveness transitions, and recovery behavior clearly enough to support operational monitoring and debugging.

Acceptance criterion. Operators can inspect status changes, replication activity, and recovery evidence through exposed membership, event, or metric views.

NFR5 – Reproducibility of deployments and validation. The platform shall support repeatable deployments and validation scenarios on development machines without relying on external managed infrastructure.

Acceptance criterion. Using the documented setup and provided Docker Compose topologies, an operator can start a predefined cluster and reproduce representative convergence, partition, and recovery scenarios across repeated runs without relying on external managed services.

NFR6 – Extensibility of runtime policies and sensor sources. The system shall be open to additional sensor producers and alternative topology policies without changing the conceptual role of the rest of the distributed runtime.

Acceptance criterion. New sensor sources or alternative topology strategies can be introduced while preserving the same downstream ingestion, replication, membership, and observation capabilities.

NFR7 – Operational scalability at intended scale. The platform shall support small and medium peer deployments rather than only a fixed two-node demo.

Acceptance criterion. The documented deployment can be instantiated with multi-node topologies and provides replicated-state and observability functions across the deployment. Validation is required at least on six-node clusters; twelve-node clusters are supported as a deployment target.

2.4 Implementation Constraints

IR1 – Python-based implementation. The project shall be implemented in Python.

Justification. Python provides a practical balance of development speed, readability, and standard-library support for networking, concurrency, HTTP serving, serialization, and testing.

Acceptance criterion. The distributed node runtime, protocol handling, failure detection, replication logic, and observability services are implemented in Python and can be executed with the documented Python environment.

IR2 – Container-based multi-node deployment support. The project shall support multi-node deployment through Docker Compose.

Justification. Docker Compose enables reproducible multi-node topologies with consistent configuration, without requiring external orchestration platforms.

Acceptance criterion. An operator can launch a documented multi-node deployment using Docker Compose and obtain a running cluster with configured node identities, networking, and observability endpoints.

IR3 – Self-contained execution on commodity machines. The project shall be executable on commodity development machines without requiring paid cloud services, institution-managed infrastructure, or external managed services.

Justification. Self-contained execution keeps deployment low-cost, accessible, and reproducible directly from repository artifacts.

Acceptance criterion. An operator can start one or more nodes using the repository code, documented dependencies, and provided deployment artifacts without provisioning external infrastructure.

3 Design

This chapter explains the strategies used to meet the requirements identified during the analysis phase (section 2). It presents the system design choices that support the functional goals and the main non-functional constraints.

3.1 Architecture

The node architecture is organized as a **layered/modular design**. The goal is to keep transport, protocol semantics, domain logic, and observability concerns explicitly separated inside each runtime instance.

At node level, the internal structure is organized into explicit layers:

- `runtime`: component composition, lifecycle orchestration;
- `domain`: `membership`, `fd`, `gossip`, `state`, `topology`, `sensors`;
- `protocol`: message contracts, codec, dispatcher, handler routing;
- `observability`: `introspection` and `webapi`.
- `networking`: TCP transport, framing, connection management, retries;

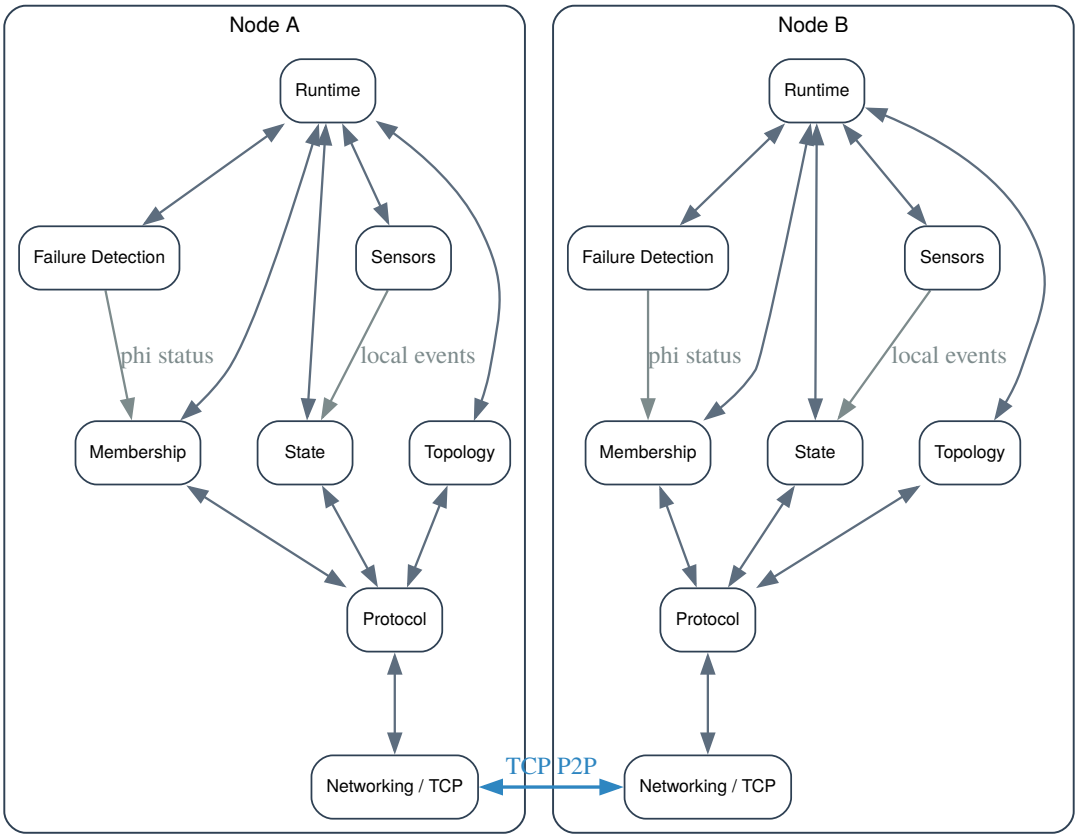

Figure 1: Internal layered architecture of a node runtime.

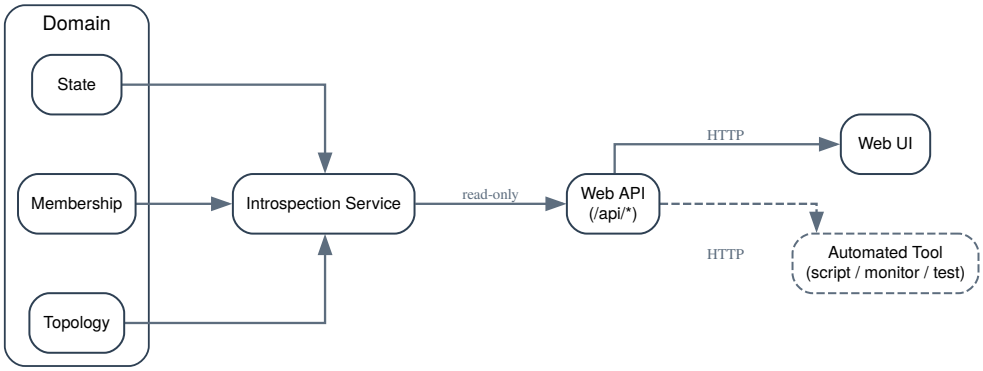

Figure 2: Observability architecture: introspection and read-only API exposure path.

This architecture separates transport concerns, protocol semantics, domain logic, and

observability concerns, improving maintainability and evolvability of the node runtime.

3.2 Infrastructure

The infrastructure is intentionally minimal and consists only of **peer nodes**. In the base deployment, the system runs with N homogeneous node processes. Each node embeds the same runtime stack (runtime, domain services, protocol handling, TCP networking, and observability endpoints), with no role specialization and no centralized infrastructure component.

Node distribution over the network follows a symmetric P2P model:

- all nodes are equivalent peers and communicate through bidirectional TCP connections;
- nodes may run on a single machine (development), within the same LAN/VPC, or across different networks and hosts;
- the only requirement is mutual reachability and TCP communication across configured inter-node endpoints.

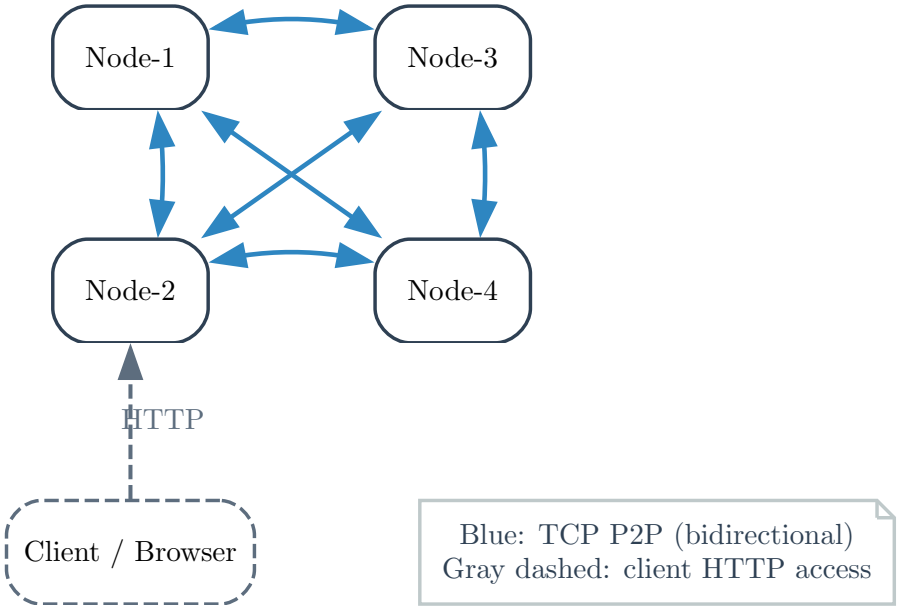

Figure 3: Infrastructure-level node connectivity in the base full-mesh deployment.

Node discovery is bootstrap-based and membership-driven:

- explicit node naming (`node-1`, `node-2`, ...) with known `host:port`;
- initial bootstrap through a static seed list in configuration;

- progressive discovery and cluster-view maintenance through membership/gossip exchange.

3.3 Modelling

At implementation level, the model is organized around node-local runtime components and their data-flow relationships.

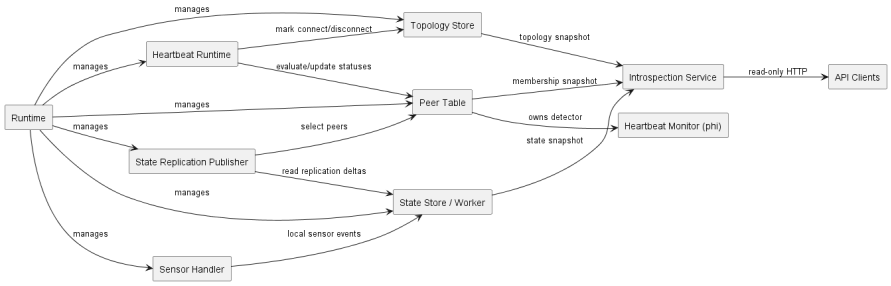

Figure 4: Domain model of node-local components and inter-component data flow.

The model includes the following components:

- **Runtime**: root orchestrator that manages the lifecycle of the internal subsystems;
- **Sensor Handler**: ingestion boundary for local sensor events;
- **State Store / Worker**: node-local replicated state authority and delta source;
- **State Replication Publisher**: component that reads replication deltas and propagates them to peers;
- **Peer Table**: local membership view of known peers and status;
- **Heartbeat Runtime**: periodic liveness loop and membership-update driver;
- **Heartbeat Monitor (ϕ)**: local failure-detection component owned by membership logic;
- **Topology Store**: local connectivity/topology view;
- **Introspection Service**: read-only aggregation endpoint;
- **API Clients**: external consumers accessing introspection over HTTP.

The main relationships represented in the model are:

- `Runtime` manages `Sensor Handler`, `State Replication Publisher`, `Heartbeat Runtime`, `Peer Table`, `State Store / Worker`, and `Topology Store`;
- `Sensor Handler` writes local sensor events to `State Store / Worker`;

- `State Replication Publisher` reads replication deltas from `State Store / Worker` and selects peers through `Peer Table`;
- `Heartbeat Runtime` evaluates/updates peer statuses in `Peer Table` and marks connect/disconnect information in `Topology Store`;
- `Peer Table` owns `Heartbeat Monitor` (ϕ);
- `State Store / Worker`, `Peer Table`, and `Topology Store` expose state/membership/topology snapshots to `Introspection Service`;
- `API Clients` consume `Introspection Service` through read-only HTTP.

This model makes explicit that authoritative operational knowledge remains node-local and in-memory, while inter-node convergence is achieved by replication and membership protocols.

3.4 Interaction

There are two main interaction families in the system.

The first is **peer-to-peer interaction among nodes**. This interaction is best-effort, message-based, and decoupled through protocol dispatch. It runs over TCP with client-side retry/queue behavior, but without end-to-end application-level delivery guarantees. It is used for:

- bootstrap and peer discovery;
- heartbeat exchange;
- dissemination of membership knowledge;
- dissemination and retrieval of replicated sensor state;
- recovery synchronization after missed updates.

The second is **observer interaction**, where external clients query a node through the HTTP read API. This interaction is request-response and read-only. In practice, the dashboard mainly relies on polling of `/api/introspection`, alongside other legacy read-only endpoints. It does not participate in cluster coordination and it never mutates the distributed state.

At the inter-node level, communication follows a layered path:

$$\textit{domain intent} \rightarrow \textit{protocol message} \rightarrow \textit{transport delivery} \rightarrow \textit{Message.decode} \rightarrow \textit{MessageDispatcher} \rightarrow \textit{handler} \rightarrow \textit{domain update}$$

The main interaction patterns are:

- **bootstrap and peer-list exchange**, used when a node joins and needs initial peer knowledge;

- **periodic heartbeat exchange**, used to collect liveness evidence;
- **gossip dissemination**, used mainly to spread membership and liveness knowledge without central coordination;
- **push-pull state synchronization**, used to exchange recent sensor-state updates efficiently;
- **full synchronization on demand**, used when incremental recovery is insufficient;
- **polling-based observation**, used by the dashboard and external tools to inspect cluster state.

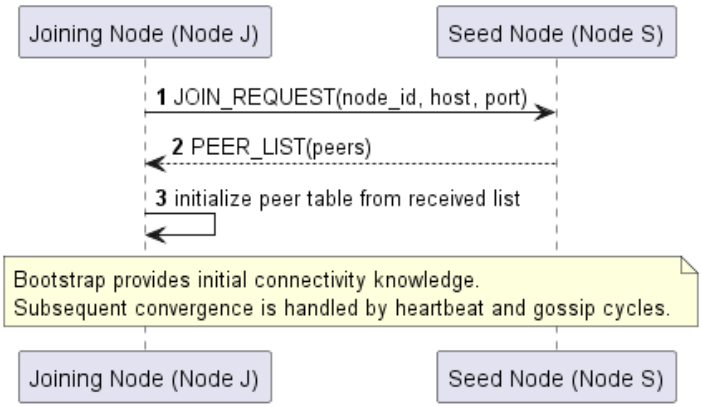

Figure 5: Bootstrap and peer-list exchange interaction for node join.

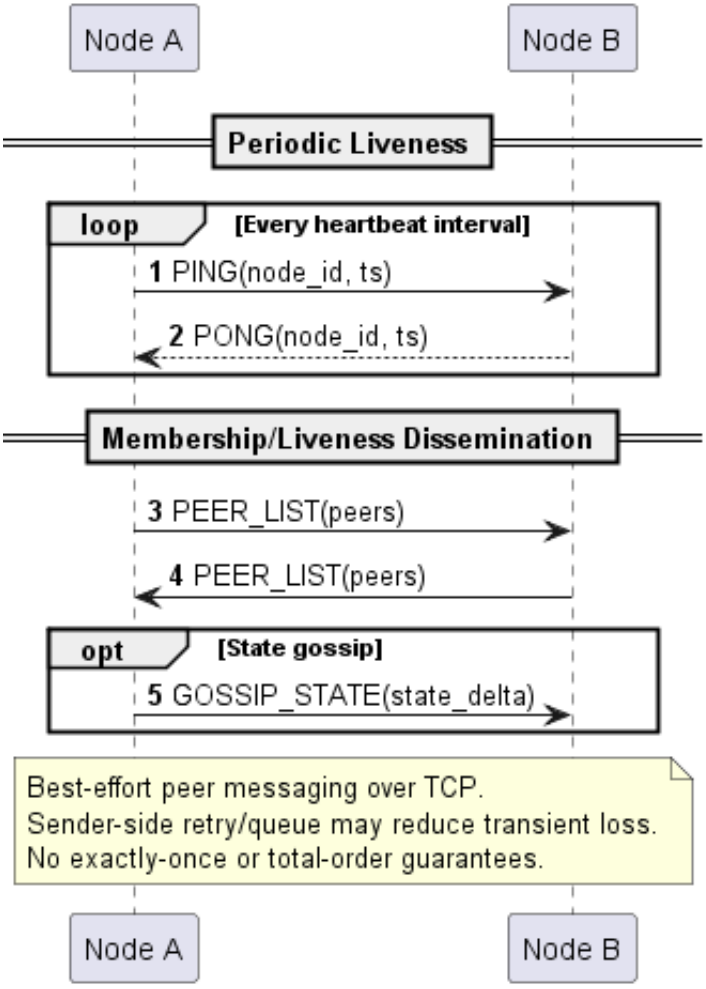

Figure 6: Inter-node heartbeat and membership-gossip interaction.

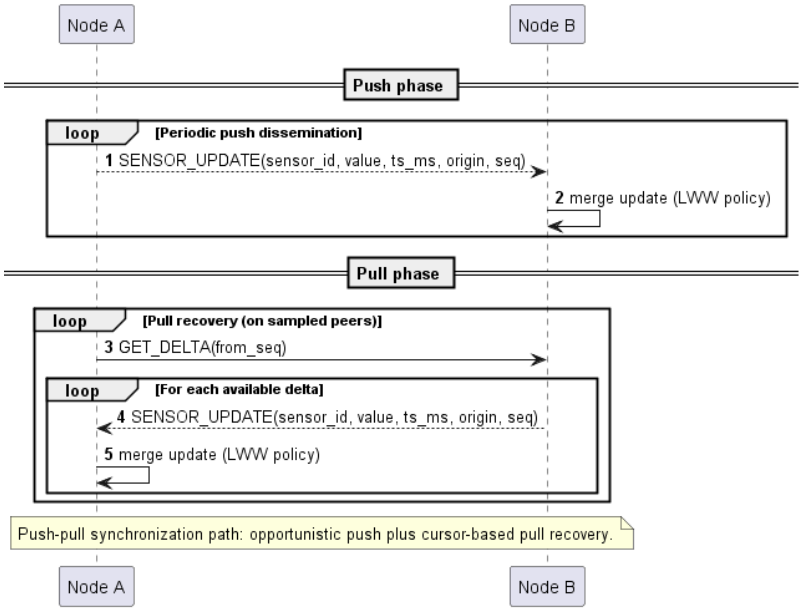

Figure 7: Incremental delta synchronization interaction.

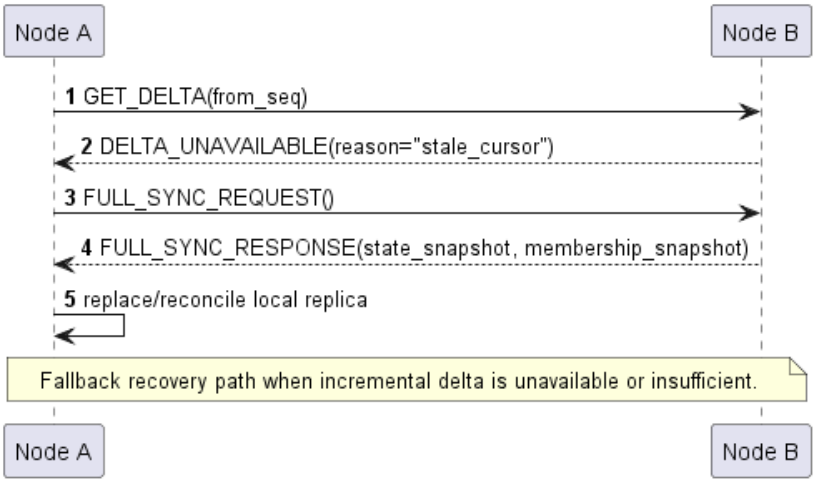

Figure 8: Full synchronization fallback when incremental delta is unavailable.

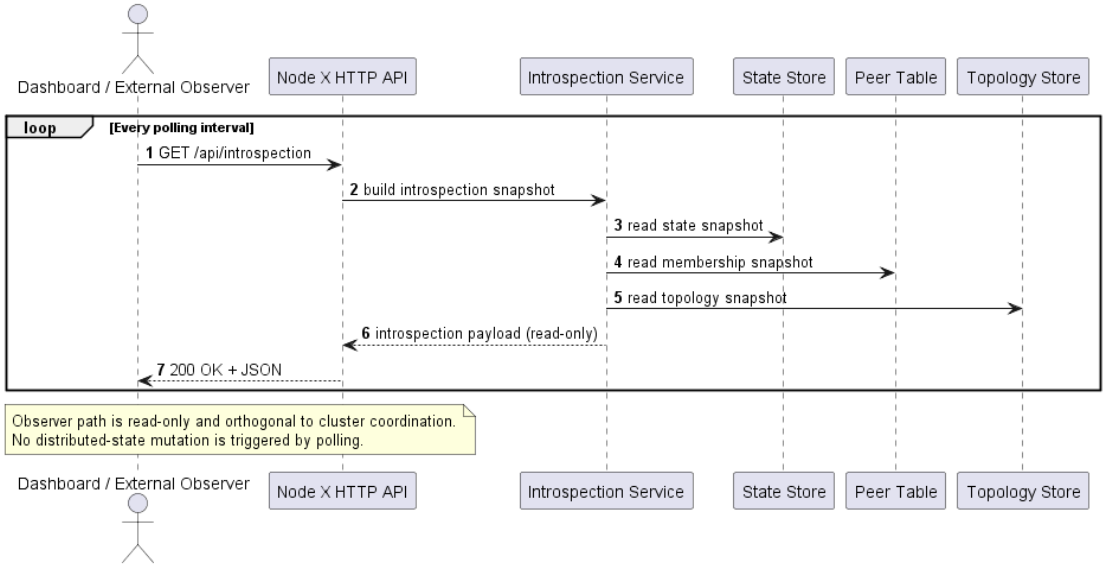

Figure 9: Observer polling over read-only HTTP introspection endpoints.

This design supports replication, decentralized membership/liveness, recovery, and read-only monitoring because it combines low-coupling peer communication with an observation interface that stays orthogonal to the replicated core.

3.5 Behaviour

The behavior of the main components can be summarized as follows.

Sensor sources are active components: they periodically generate local readings and inject them into the node. They are conceptually producers and do not need global knowledge.

The local state manager is stateful and authoritative for the node's replica. More precisely, it is authoritative for local Last-Write-Wins winners and for the node's replication-delta view. It receives local and remote updates, applies deterministic merge rules, stores the current winners, and exposes snapshots to both replication and observability components. This component is central to ingestion, convergence, and deterministic conflict resolution.

The membership manager is also stateful. It stores known peers, updates direct or indirect liveness evidence, and maintains the node's local view of cluster membership.

The failure detector behaves as a local evaluator rather than as a replicated authority. It interprets heartbeat timing and produces suspicion levels for peers observed directly by the node. Those local judgements are then translated into membership information that can be disseminated.

The heartbeat and gossip runtime is periodic. At each round, it evaluates liveness, sends heartbeat probes, and disseminates updated membership knowledge. Its purpose

is not to guarantee exact membership synchronization at every instant, but to make peer-status knowledge progressively converge.

The replication runtime is periodic as well. It pushes recent sensor-state updates to selected peers and pulls missing updates from others. In particular, the publisher side targets peers currently considered `alive`, while heartbeat and membership gossip activities may still attempt communication toward peers that are not currently classified as `alive`. When incremental recovery is not sufficient, it can trigger a broader synchronization phase. This behavior is essential for convergence, recovery, and partition-aware operation.

The observability subsystem is read-only. It collects snapshots from state, membership, topology, and event/metric providers, then publishes a coherent observation view for users and tools.

State updates are therefore concentrated in a small set of stateful components:

- local sensor-state storage;
- local membership storage;
- local topology and observability stores.

Other components mainly trigger transitions or move data between these stores. This separation keeps write paths explicit and supports maintainability.

3.6 Data and Consistency Issues

The system stores several forms of node-local runtime data:

- the replicated sensor-state winners;
- the incremental replication view needed for propagation and recovery;
- the membership table and liveness metadata;
- topology knowledge;
- recent events and metrics for introspection.

This data is primarily **runtime state**, not long-term persistent storage. The design does not require an external database because the goal is to study distributed replication and convergence of current system knowledge rather than durable business records. This keeps the architecture lightweight and self-contained, matching IR3.

Therefore, the current design has no durable persistent dataset. If persistence were introduced in a future version, the natural model for sensor winners would be a key-value representation keyed by logical sensor identifier, while event and metric history would more naturally be stored as append-oriented records for later inspection.

Accordingly, no component performs database queries in the present design, because no database is part of the architecture. Instead, nodes read environment-backed configuration, node-local in-memory state, and data received from peers over the inter-node protocol, while observers query read-only HTTP snapshots exposed by each node.

Conceptually, the replicated sensor state behaves like a key-value map from a logical sensor identifier to the current winning record. Internally, records are indexed by logical `sensor_id`, while API snapshots expose them using the derived `origin:sensor_id` form so that values from different origins remain unambiguous to observers. The important property is not relational querying power, but deterministic merge semantics. For this reason, the core consistency problem is solved at the application level rather than delegated to a storage engine.

Consistency is handled differently for different data types:

- **sensor state** uses eventual consistency with deterministic Last-Write-Wins conflict resolution based on timestamp and origin metadata;
- **membership state** also converges eventually, but it reflects local observations that may temporarily differ across nodes;
- **topology state** converges eventually through gossip and uses per-node Last-Write-Wins declarations based on update timestamps, with deterministic tie-breaking for peer lists;
- **observability data** is inherently approximate and time-sensitive, because it represents a snapshot of changing distributed state.

This means the system explicitly rejects strong consistency as a primary goal. During partitions or transient delays, different nodes may expose different but still valid local views. Once communication resumes and no newer conflicting updates appear, the replicas converge. This choice is aligned with availability and partition tolerance.

Data is shared among components in well-defined ways:

- sensor producers share generated readings with the local state manager;
- the state manager shares snapshots and deltas with the replication runtime and the observability subsystem;
- the membership subsystem shares peer-status information with heartbeat, gossip, topology, and observability components;
- the observability subsystem reads from other stores but does not modify them.

This read/write asymmetry is deliberate: observation must remain decoupled from mutation without introducing accidental feedback paths.

Concurrent access to local state is expected because sensor ingestion, TCP protocol handlers, replication publishing, heartbeat evaluation, and HTTP polling may run at the same time. The design therefore separates write ownership and read access. Local sensor producers do not conceptually mutate the replicated state directly: they emit readings into an ingestion path consumed by the local state manager. Remote updates and full-sync responses also enter through explicit merge operations, while observers receive snapshots rather than direct references to mutable stores.

The consequence is that concurrent reads and writes are allowed, but they are confined to node-local stores and protected by implementation-level synchronization. There is no distributed transaction spanning sensor state, membership, topology, and observability data. This limitation is acceptable because these views are intended to be eventually consistent, approximate, and inspectable rather than globally atomic.

3.7 Fault-Tolerance

Fault tolerance is one of the central design drivers of the project. The first mechanism is **state replication**: sensor-state knowledge is disseminated among peers so that the system does not depend on a unique owner of cluster knowledge. Replication is best-effort and decentralized, preserving autonomy and partition resilience even though it does not provide instantaneous consistency.

The second mechanism is **heartbeat-based liveness observation**. Nodes periodically exchange liveness probes and evaluate the timing of received heartbeats. This allows each node to form a local judgement about whether a peer is alive, suspected, or dead. Since such knowledge is local and timing-dependent, the design treats liveness as a continuously revised estimate rather than as an absolute truth.

Timeout semantics are therefore handled by the interaction between the heartbeat runtime, the failure detector, and the membership subsystem: missing or delayed heartbeat evidence increases suspicion, and the resulting status change is reflected in the local membership view rather than blocking the node.

The third mechanism is **membership gossip**. A node does not keep liveness knowledge only for itself; it disseminates updated membership information so that peer knowledge can propagate even when direct connectivity patterns are incomplete. This improves resilience of membership awareness and helps the system maintain a broader view of the cluster.

The fourth mechanism is **anti-entropy recovery**. Incremental propagation is efficient when peers stay mostly synchronized, but it may be insufficient after outages, restart, or missed traffic. For this reason, the design includes a stronger recovery path that allows a lagging node to regain a coherent replicated view after disruption.

Retry behavior is present mainly as periodic or protocol-level reattempt rather than as a separate user-visible subsystem. Connection establishment can be retried with bounded backoff, heartbeat and replication rounds naturally re-attempt communication over time, and missed information can later be recovered through incremental synchronization or, when incremental deltas are no longer available, through a full synchronization exchange.

Communication errors are treated as local, recoverable events. A failed send, missed heartbeat reply, or temporarily unreachable peer does not stop sensor ingestion or HTTP observation on the affected node; instead, the peer's membership status can degrade, later protocol rounds can retry communication, and synchronization can repair missed state when a route becomes available again.

Error handling is intentionally failure-aware rather than failure-oblivious. If a component or peer becomes temporarily unavailable:

- surviving nodes continue local operation;
- local replicas may diverge temporarily;
- membership views reflect degraded reachability;
- synchronization resumes when communication becomes possible again.

This behavior is preferable to halting the whole system because the project prioritizes availability and observation under faults over strong global coordination.

3.8 Availability

The design does not introduce classical web-style caching layers or external load balancers, because the system is not centered on high-volume client traffic toward a centralized service. Instead, availability is achieved structurally through replication, autonomy of nodes, and local read access to each node's current view.

Each node acts as a locally available observation point. As long as a node remains running, it can still:

- ingest its own local sensor readings;
- maintain its local replica;
- expose read-only snapshots to operators and tools.

In this sense, replicated state itself plays a role analogous to availability-oriented caching: each node keeps a local, eventually convergent working copy of shared cluster knowledge, so read operations do not require contacting a central authority.

No explicit load balancing is required because there is no single mandatory ingress point for observation. Different observers may query different nodes, and each node exposes its own local view. This is sufficient for the intended scale of the project.

During a **network partition**, the design favors continued local operation over blocking. Nodes inside each connected component continue to exchange information among themselves, while nodes separated by the partition stop receiving fresh updates from one another. Consequently:

- sensor-state replicas may diverge temporarily across partitions;
- membership views may mark remote peers as suspected or dead;
- local observation remains available inside each connected component.

When the partition heals, incremental dissemination and recovery mechanisms progressively re-align the replicas. This behavior is a direct consequence of choosing availability and partition tolerance over strong consistency.

3.9 Security

Security is intentionally limited in the current design because, as discussed in section 2.1, it is not a primary driver of this project. The system is assumed to run in controlled laboratory, development, or trusted demonstration environments.

Accordingly, the current design does **not** make authentication a core architectural dependency for either peer-to-peer traffic or read-only observation endpoints. Similarly, it does not define a rich authorization model with differentiated user permissions. The effective roles are simple: nodes participate in the distributed protocol, while observers query read-only information.

The corresponding effective access rights are therefore minimal and explicit. Peer nodes may exchange internal protocol messages, maintain membership and liveness information, and merge received updates into their own local replicated state. External observers and clients are limited to read-only HTTP observation, since no write-oriented or administrative HTTP API is exposed by the current design.

This choice keeps the architecture aligned with the educational scope of the project and avoids obscuring the main distributed-systems concerns with security infrastructure that is orthogonal to the report goals. However, the design has been kept modular enough that stronger security policies could be introduced later at clear boundaries:

- on node-to-node channels, to authenticate peers and protect inter-node communication;
- on HTTP observation endpoints, to restrict access to monitoring data;
- on deployment boundaries, through network isolation and reverse-proxy policies.

Likewise, no cryptographic protocol is a conceptual prerequisite of the current system model. Confidentiality and strong identity guarantees are therefore outside the present scope and remain future extensions rather than core design commitments.

4 Implementation

This chapter reports the concrete technology-dependent choices used to realize the design discussed in section 3. The implementation deliberately favors a lightweight and inspectable stack, so that the distributed behavior of the system remains visible and reproducible without relying on heavy external infrastructure.

4.1 Protocols and Data Representation

Two communication protocols are used, each chosen according to the interaction style required by the system.

Node-to-node communication. Peer communication is implemented over **TCP**. This choice was preferred over UDP because the project needs ordered and reliable byte-stream delivery while a connection remains healthy, while still keeping the communication stack simple and under direct control. TCP is therefore used for:

- bootstrap and peer discovery traffic;
- heartbeat exchange;
- membership gossip dissemination;
- sensor-state update propagation;
- incremental and full synchronization during recovery.

Rather than adopting an external RPC or messaging framework, the project uses a custom lightweight application protocol on top of TCP. Payloads are transmitted as length-prefixed frames, which makes message boundaries explicit and keeps the implementation compatible with a persistent stream-oriented transport.

Observer communication. Human users and external tools interact with the system through a **read-only HTTP API**. HTTP is appropriate here because the observability plane is request-response oriented, browser-friendly, and naturally suited to dashboards, integration scripts, and test harnesses. Since the observation API does not participate in peer coordination, it is intentionally separated from the internal peer-to-peer protocol.

In-transit representation. Messages and HTTP responses are represented as **JSON**. This choice was motivated by readability, ease of debugging, and direct interoperability with browsers and Python tooling. JSON is sufficient for the project because payload sizes are moderate and the main goal is clarity rather than binary compactness. Message validation and dispatch are then handled at the protocol layer after decoding.

The main message families implemented in the peer protocol are:

- `JOIN_REQUEST` and `PEER_LIST` for discovery;
- `PING` and `PONG` for heartbeat-based liveness observation;
- `GOSSIP_STATE` for disseminating membership and topology knowledge;

- `SENSOR_UPDATE` for push-based sensor replication;
- `GET_DELTA` and `DELTA_UNAVAILABLE` for incremental recovery;
- `FULL_SYNC_REQUEST` and `FULL_SYNC_RESPONSE` for stronger catch-up after disruption.

4.2 State Management and Storage Choices

The implementation does **not** use an external database. All relevant information is maintained as in-memory runtime state inside each node:

- the current winning sensor records;
- replication deltas and sequence metadata;
- the peer membership table;
- liveness-related information;
- topology snapshots;
- introspection events and counters.

This is consistent with the project scope: the goal is to study replication, convergence, recovery, and observability of current distributed state, not durable long-term business storage. As a consequence, no SQL or NoSQL query layer is required. Queries are replaced by direct access to in-memory state providers, which are then exposed through the HTTP observation endpoints.

From an implementation viewpoint, the most important storage choice is therefore not the selection of a database technology, but the decision to encode merge semantics explicitly in the application logic. Sensor-state convergence is implemented with deterministic Last-Write-Wins conflict resolution based on timestamp and origin metadata, while membership and liveness views are maintained through dedicated in-memory structures updated by protocol handlers and periodic runtimes.

4.3 Authentication and Authorization

No explicit authentication or authorization protocol is implemented in the current version of the system. This is a deliberate scope decision, not an omission by accident. The project runs in controlled development and laboratory environments, and the report prioritizes decentralization, consistency, failure handling, and observability over access-control concerns.

Accordingly:

- peer-to-peer channels do not currently require node authentication;
- the HTTP API does not require login, tokens, or session management;

- the system does not implement role-based or attribute-based authorization.

The practical effect is that any process able to reach a node in the current deployment can interact with the exposed interface allowed by that channel. This is acceptable for the project scope, but it also marks a clear limitation of the current implementation. Stronger authentication, endpoint protection, and transport hardening are left as future work rather than embedded in the present runtime.

4.4 Technological details

Programming language and runtime. The project is implemented in **Python**. This choice supports rapid experimentation, readable code, and low setup cost for a course project. The implementation makes extensive use of the Python standard library for networking, concurrency, HTTP serving, serialization, logging, and timing, which keeps external dependencies intentionally minimal.

Concurrency model. The runtime is based on **threads**, background worker loops, and in-memory queues. This model is used for several long-lived activities that must proceed concurrently:

- accepting inbound TCP connections;
- maintaining outbound peer workers;
- periodic heartbeat and failure-detector evaluation;
- periodic push-pull replication;
- sensor generation;
- HTTP serving for observation.

This approach was preferred to a more complex actor framework or asynchronous event-loop architecture because it keeps the execution model explicit and understandable, which is desirable for an educational distributed-systems project.

Concurrent access to local replicated state is handled through a combination of queue-based ingestion and explicit locking. Local sensor providers enqueue normalized sensor events into a thread-safe event queue consumed by the `NodeStateWorker`; remote protocol handlers can invoke merge operations for `SENSOR_UPDATE` and `FULL_SYNC_RESPONSE` messages. The underlying `NodeStateStore` protects sensor records, update buffers, replication deltas, sequence cursors, and origin-sequence tracking with a mutex. Membership and failure-detector state are protected separately by locks in the peer table and heartbeat monitor. This mitigates in-process race conditions among sensor generation, TCP handlers, replication publishing, heartbeat evaluation, and HTTP polling. The remaining limits are deliberate: state and membership are not updated atomically as one transaction, replication buffers are bounded and volatile, and recovery relies on peer synchronization rather than durable local storage.

Configuration management. Node configuration is externalized through **environment variables**. Parameters such as node identity, ports, bootstrap peers, sensor definitions, replication cadence, failure-detector thresholds, and fault-injection options are all configurable from the environment. This makes it easy to instantiate different topologies and experimental scenarios without changing source code.

Sensor implementation. Sensor sources are implemented as pluggable local producers. The repository includes multiple simulated sensor types, such as numeric, boolean, wave, noise, spike, trend, categorical, incremental, and finite test sensors. This confirms the extensibility objective identified in NFR6 while keeping the whole platform reproducible in the absence of real hardware devices.

Networking layer. The transport layer is implemented with Python sockets and a custom framed TCP protocol. On the outbound side, each peer connection has its own worker and FIFO send queue; on the inbound side, a threaded TCP server accepts connections, reads framed messages, validates them, and dispatches them to protocol handlers. The implementation also includes bounded worker counts and configurable socket limits, which helps prevent unbounded resource growth in larger experiments.

HTTP observation layer. The Web API is implemented with Python's `http.server` facilities, in particular a threaded HTTP server and custom request handlers. This is sufficient because the API is read-only and relatively small, so introducing a larger web framework would not provide proportionate benefits for the project scope.

Containerization and deployment automation. The project uses **Docker** and **Docker Compose** to instantiate reproducible node topologies. Multiple compose configurations are provided, including small and medium cluster layouts, so that the same codebase can be exercised under different scales and fault scenarios. Containerization strongly supports NFR5 because it reduces environmental variability across test runs and demonstrations.

Testing and quality tools. Validation and development are supported by:

- **pytest** for unit and integration testing;
- Docker-backed integration harnesses for multi-node and fault-recovery scenarios;
- **pyright** for static type checking;
- structured logging and introspection endpoints for runtime diagnosis.

Dependencies. External dependencies are intentionally few. Besides development tools, the main runtime dependency is `python-dotenv`, used to simplify environment-based configuration. This minimal dependency footprint is coherent with the project's emphasis on portability, inspectability, and self-contained deployment.

5 Validation

Project validation was designed across multiple layers to verify both the functional correctness of individual modules and the emergent behavior of the distributed system under replication, failures, and reconnection. The adopted strategy combines unit tests, integration tests, smoke tests in a containerized environment, and manual observation scenarios, with the goal of covering the entire node lifecycle: startup, bootstrap, state propagation, failure detection, recovery, and introspection.

5.1 Automatic Testing

Unit tests. The foundation of the validation plan is a unit test suite developed with `pytest`. Unit tests verify, in isolation, the responsibilities of the system's main modules:

- `tests/protocol/`: correctness of serialization, deserialization, validation, and dispatch of application-protocol messages;
- `tests/networking/`: TCP framing, client/server connection handling, operational limits, and transport robustness;
- `tests/state/`: deterministic application of the Last-Write-Wins policy and maintenance of incremental deltas;
- `tests/membership/` and `tests/fd/`: membership-view updates, heartbeat handling, and *phi-accrual* estimation for peer classification;
- `tests/sensors/`: behavior of simulated sensor generators and the related lifecycle manager;
- `tests/webapi/` and `tests/introspection/`: consistency of read-only HTTP endpoints and of the introspection contract exposed to clients.

The rationale for this suite is twofold: on the one hand, to prevent regressions in components with local semantics; on the other hand, to make explicit the invariants that the system must preserve regardless of distributed topology. At the time of local repository verification, running `python -m pytest -m "not integration" -q` produced `180 passed, 9 deselected`, confirming the stability of the non-integrated portion of the suite.

In-process integration tests. A second validation layer exercises interaction among multiple real nodes executed within the same test process or in processes coordinated by dedicated harnesses. In particular:

- `tests/integration/test_deterministic_state_convergence.py` verifies replicated-state convergence in a six-node cluster, checking that final LWW winners match expected outcomes;
- `tests/integration/test_cluster_harness.py` validates the correct behavior of the supporting infrastructure for reproducible distributed scenarios;

- `tests/integration/test_finite_test_sensor.py` uses deterministic sources to make update sequences observable and comparable across nodes.

These tests cover properties that cannot be inferred from local correctness alone: incremental propagation, relative ordering of updates, initial bootstrap, and eventual convergence after multiple message exchanges.

Docker-based integration tests and production-like scenarios. The third validation layer uses Docker Compose topologies to exercise the software in an environment closer to real deployment. This category includes:

- `tests/integration/test_docker_deterministic_state_convergence.py`, which replicates convergence verification on containerized nodes;
- `tests/integration/test_docker_crash_restart_recovery.py`, which evaluates recovery after node stop and restart;
- `tests/integration/test_docker_partition_reconciliation.py`, which verifies state reconciliation after a connectivity discontinuity;
- `tests/integration/verify_cluster.py`, used as an end-to-end smoke test to check correct cluster startup and reachability of primary endpoints.

The main advantage of this solution is that the same deployment setup used for demonstrations is reused as test infrastructure. This reduces the distance between development, verification, and presentation environments, improving the reproducibility of experiments.

Execution automation. Automated test execution is also supported by CI workflows defined in `.github/workflows/unit-tests.yml` and `.github/workflows/integration-tests.yml`. The unit-test pipeline installs dependencies and runs `pytest` while excluding integration tests, whereas the integration pipeline runs both deterministic convergence checks and a smoke test on the `docker/docker-compose-base.yml` Docker topology. This automation helps detect regressions in both local logic and the most relevant distributed scenarios.

How to run the tests. The main execution modes, already documented in the repository, are the following:

- unit and modular tests: `pytest --maxfail=1` or `python -m pytest -v -m "not integration"`;
- protocol tests: `pytest -m protocol`;
- targeted verification of LWW convergence:

  ```
  python -m pytest tests/integration/test_deterministic_state_convergence.py -q -s
  ```

- Docker end-to-end smoke test for a 6-node manual scenario: start the Compose topology with:

```
docker compose -f docker/docker-compose-6-nodes.yml up --build -d
```

then run:

```
python tests/integration/verify_cluster.py --timeout 60 --interval 2
```

Coverage with respect to project goals. Overall, automated tests cover the most important points identified in the project requirements:

- correctness of the communication protocol and its validations;
- deterministic convergence of replicated state;
- propagation of membership and liveness information;
- tolerance to crashes, restarts, and temporary partitions;
- external verifiability through HTTP APIs and introspection.

More explicitly, the main test groups map to requirements as follows:

`tests/state/test_lww.py` verifies Last-Write-Wins merge behavior, stale-update rejection, tie breaking, full-state merge, partition convergence, delta buffers, and cursors. It covers FR03, FR04, FR06, NFR2, and NFR3.

`tests/fd/` **and** `tests/membership/` verify phi estimation, peer-table updates, heartbeat transitions, suspected/dead/alive recovery, gossip status merging, and membership snapshots. They cover FR05 and NFR4.

`tests/protocol/test_full_sync_handlers.py` verifies `GET_DELTA`, `DELTA_UNAVAILABLE`, full-sync requests, full-sync responses, and recovery fallback. It covers FR03, FR06, and NFR2.

`tests/protocol/test_gossip_state_handlers.py` verifies membership gossip, stale status rejection, delayed convergence after partitions, and topology merge. It covers FR05, NFR1, NFR3, and NFR4.

`tests/runtime/test_sensor_update_publisher.py` verifies push-pull fanout, alive-peer targeting, and pull tracking. It covers FR03, NFR2, and NFR7.

`tests/webapi/` **and** `tests/introspection/` verify read-only HTTP endpoints and aggregate introspection snapshots. They cover FR05, FR07, FR08, and NFR4.

`tests/sensors/` verifies simulated sensor providers and sensor-manager configuration. It covers FR02 and NFR6.

`tests/networking/` verifies TCP framing, connection handling, concurrency, slow clients, bursts, limits, and shutdown. It covers FR01, FR03, NFR1, and NFR7.

`tests/topology/` verifies topology-state merge and full-mesh policy resolution. It covers FR05, FR07, NFR7, and IR3.

`tests/integration/test_deterministic_state_convergence.py` verifies that six real in-process nodes converge to deterministic expected winners. It covers FR01, FR02, FR03, FR04, NFR1, NFR2, NFR7, and IR2.

`tests/integration/test_cluster_harness.py` verifies that the reusable cluster harness starts nodes and exposes state and membership. It covers FR01, FR05, FR07, IR1, and IR2.

`tests/integration/test_docker_deterministic_state_convergence.py` verifies convergence in container networking. It covers FR01, FR02, FR03, FR04, NFR2, NFR5, NFR7, IR1, and IR2.

`tests/integration/test_docker_crash_restart_recovery.py` verifies crash/restart recovery, state reconciliation, and membership recovery. It covers FR05, FR06, NFR3, NFR4, and NFR5.

`tests/integration/test_docker_partition_reconciliation.py` verifies temporary network partition, divergence, healing, and reconciliation. It covers FR03, FR04, FR06, NFR2, and NFR3.

`tests/integration/verify_cluster.py` performs a Docker smoke check that each node exposes replicated data from expected origins. It covers FR01, FR03, FR07, NFR5, IR1, and IR2.

The rationale of the main distributed tests is also requirement-driven. Deterministic convergence tests prove that autonomous nodes exchange finite sensor streams and converge to the same Last-Write-Wins result, intercepting silent replication failures, incorrect fanout, or non-deterministic merge behavior. Docker convergence repeats the same idea in container networking, where DNS names, exposed ports, startup timing, and network isolation are closer to deployment conditions. Crash/restart recovery validates that a restarted node, after losing volatile in-memory state, can catch up through delta exchange or full synchronization, while surviving peers continue operating. Partition reconciliation checks that the system can remain locally available during a split, accept temporary divergence, and converge again after communication is restored. Full-sync handler tests specifically protect against the risk that stale cursors make recovery impossible, while web API and introspection tests protect the dashboard and external tools from unstable or incomplete observation contracts.

The validation does not attempt to prove formal correctness, Byzantine tolerance, security properties, or durable recovery after full cluster loss. These aspects are outside the current project scope.

5.2 Acceptance test

Alongside automated validation, manual acceptance tests were also executed, primarily focused on the system's observable and experimental aspects. These tests were performed through:

- startup of base, 6-node, and 12-node Docker clusters;
- direct inspection of `/api/state`, `/api/membership`, `/api/introspection`, and related derived views;
- use of the node-served dashboard (for example `http://localhost:10000/ui`) to observe topology, recent events, replication metrics, and peer status;
- execution of `manual_tests/compose_chaos.py` to selectively stop and restart services during cluster execution.

These manual tests were intended to confirm properties that are difficult to capture fully through automated assertions alone: readability of exported state, quality of observability, temporal consistency of dashboards, and perceivable system behavior during reconfiguration and network transients.

The manual component does not replace the automated suite, but complements it. In particular, it proved useful when acceptance criteria were also qualitative, for instance in evaluating the clarity of information shown to users or in visually diagnosing phenomena such as pull storms, accumulated delays, or rapid membership state transitions.

6 Deployment

Project deployment was designed with two complementary objectives: enabling rapid execution on a single machine for development and debugging activities, while ensuring reproducibility of multi-node topologies through containerization. System distribution does not require external infrastructure, dedicated orchestrators, or persistent supporting services: each node autonomously executes its own networking logic, replication, failure detection, and HTTP observability.

6.1 Software prerequisites

To run the project from scratch, the following tools are required:

- a recent version of **Python 3**; the repository CI pipeline explicitly validates execution with `Python 3.12`;
- **pip** for Python dependency installation;
- **Docker** and **Docker Compose** for launching containerized topologies;
- optionally, a web browser to inspect the observability dashboard and HTTP endpoints exposed by nodes.

Runtime dependencies are intentionally minimal and defined in `requirements.txt`; development and static-analysis tools are instead collected in `requirements-dev.txt`, while `package.json` supports npm-based execution of `pyright`.

6.2 Local installation

Base project installation follows a linear procedure:

1. clone the repository;
2. enter the project directory;
3. install required Python dependencies.

A possible command sequence is the following:

```
git clone https://github.com/AlexSantini10/distributed-sensor-hub.git
cd distributed-sensor-hub
pip install -r requirements.txt
```

At the end of the installation, the repository is ready both for direct execution of a single node and for running automated tests. A recommended initial check is:

```
python -m pytest -m "not integration" -q
```

This verifies local environment consistency immediately.

6.3 Configuration through environment variables

The entire node operational configuration is externalized through environment variables. This approach makes deployment easy to reproduce both locally and in containers, avoiding dependence on rigid configuration files and allowing different topologies to be described without code changes.

The `.env.example` file documents the main variables, including:

- node identification and binding: `NODE_ID`, `HOST`, `PORT`, `WEB_API_PORT`;
- initial bootstrap: `BOOTSTRAP_PEERS`;
- TCP server limits: `SOCKET_TIMEOUT`, `ACCEPT_QUEUE_SIZE`, `MAX_CONNECTIONS`, `MAX_WORKERS`;
- failure detector parameters: `PHI_THRESHOLD_SUSPECT`, `PHI_THRESHOLD_DEAD`, `PHI_INITIAL_INTERVAL_S`
- push/pull replication cadence: `GOSSIP_SYNC_INTERVAL_MS`, `GOSSIP_PUSH_*`, `GOSSIP_PULL_*`;
- simulation of network delay and packet loss: `NETWORK_DELAY_*`, `NETWORK_PACKET_LOSS_PROB`;
- definition of local sensors: `SENSORS` and the `SENSOR_<i>_*` family.

The practical outcome is that the same software image can assume different roles simply by changing the variable set, a useful feature for both testing and classroom demonstrations.

6.4 Running a single node

For a minimal local run, it is sufficient to define a few essential variables and start `node.py`. A PowerShell example is:

```
$env:NODE_ID="node-1"
$env:HOST="0.0.0.0"
$env:PORT="9000"
$env:BOOTSTRAP_PEERS=""
$env:WEB_API_PORT="10000"
$env:LOG_LEVEL="INFO"
$env:LOG_FILE="logs/node-1.log"
python node.py
```

The expected result is startup of the node with active TCP listener and HTTP server. In this mode the system is already queryable through web APIs, for example on `/api/state`, `/api/membership`, and `/api/introspection`.

6.5 Containerized deployment

Multi-node deployment is based on Docker Compose. The repository includes multiple Compose files, each targeting a different experimental scenario:

- `docker/docker-compose-base.yml`: minimal 2-node topology, quick to start, useful for smoke tests and quick demos;
- `docker/docker-compose-6-nodes.yml`: intermediate scenario, suitable for clearly observing state convergence and gossip dynamics;
- `docker/docker-compose-12-nodes.yml`: denser scenario, designed to stress dissemination and observability readability;
- Compose files dedicated to integration tests: used for deterministic scenarios and automated validation.

Startup is performed with commands such as the following, choosing one topology at a time because the scenarios expose overlapping container names and host ports:

```
docker compose -f docker/docker-compose-base.yml up --build -d
docker compose -f docker/docker-compose-6-nodes.yml up --build -d
docker compose -f docker/docker-compose-12-nodes.yml up --build -d
```

The expected result is the creation of multiple homogeneous containers, each with:

- its own logical identity (`NODE_ID`);
- distinct TCP and HTTP ports exposed to the host;
- dedicated sets of simulated sensors;
- independently configured network and replication parameters;
- a shared log directory mounted as a volume for external inspection.

6.6 Engineering of Compose files

The `docker-compose` files do not merely instantiate identical node copies; they encode lab scenarios with heterogeneous profiles. In particular:

- bootstrap peers are distributed across nodes to avoid dependence on a single initial contact point;
- sensors differ by type, periodicity, latency, and measurement unit, so as to simulate non-uniform workloads;
- some nodes deliberately introduce higher delays, jitter, or packet-loss probabilities to make degradation and recovery phenomena observable;
- replicated-delta buffer size and push/pull thresholds are explicitly encoded in deployment, facilitating experimental tuning.

This choice is methodologically relevant: deployment is an integral part of the distributed experiment, not a mere operational detail. Compose files therefore function both as execution tools and as reproducible descriptions of scenarios analyzed during validation.

6.7 Startup verification

After deployment, verification can be conducted at three levels:

- infrastructure level, by checking container status with `docker compose ps`;
- application level, by querying HTTP endpoints on one or more nodes;
- system level, by running `python tests/integration/verify_cluster.py` to confirm startup, reachability, and baseline cluster availability.

In this way, deployment is not considered complete only when processes are active, but when the cluster effectively exhibits the expected distributed behavior.

6.8 Continuous delivery of the Docker image

The project also includes a versioned continuous-delivery path for Docker images. The Docker CD workflow is triggered on pushes to the `main` branch, pull requests targeting `main`, manual execution, and version tags matching the `v*` pattern.

For normal pushes to `main`, the workflow builds the node image from `docker/Dockerfile.base` and publishes it to GitHub Container Registry. The image receives rolling tags such as `latest`, `main`, and a commit-based `sha-*` tag. This makes the most recent development version easy to pull while still preserving a commit-specific image reference.

Versioned delivery happens when a Git tag such as `v1.0.0` is pushed. In that case, the same workflow also publishes a Docker image tagged with the version name and creates a GitHub Release. The release contains the immutable image digest and a small operational bundle: a release note file, an example environment file for running one node, a minimal Docker Compose file pinned to the exact image digest, and a short README.

This separates rolling deployment from explicit version publication. The `main` and `latest` tags are convenient for continuous development, while version tags provide reproducible release points. Because the release Compose file references the image by digest, a user can recreate the exact container image associated with that source version rather than depending on a mutable tag.

7 User Guide

This guide explains how to run the system, verify expected behavior, and collect evidence from the API and UI.

7.1 How to use the software (instructions)

Recommended demo setup (12 nodes), from the project root directory:

```
docker compose -f docker/docker-compose-12-nodes.yml up --build -d
```

Open the dashboard on any node using:

```
http://<node-ip>:<WEB_API_PORT>/ui
```

For the `docker/docker-compose-12-nodes.yml` setup, use `http://localhost:10000/ui` (node-1).

Useful API checks (example on node-1, `localhost:10000`):

```
curl http://localhost:10000/api/membership
curl http://localhost:10000/api/state
curl http://localhost:10000/api/introspection/metrics
```

Stop the cluster:

```
docker compose -f docker/docker-compose-12-nodes.yml down
```

7.2 Expected outcomes

After startup and a short convergence interval, the expected outcomes are:

- `/api/membership`: peers visible for the active topology (12 nodes in this setup);
- `/api/state`: non-empty replicated sensor state, similar across nodes;
- `/api/introspection/metrics`: non-zero gossip/replication activity;
- dashboard (`/ui`): topology graph, peer liveness (`alive`/`suspected`/`dead`), sensor table, and recent events rendered without extra services.

If `get_delta_unavailable_total` grows continuously, the delta buffer may be too small for current load or rejoin timing.

7.3 Screenshots (report evidence)

Capture screenshots from a real run of the 12-node topology.

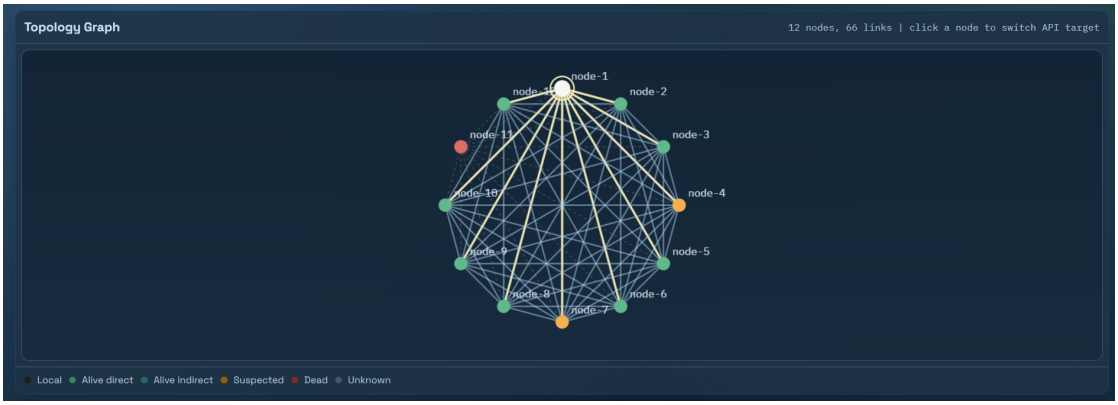

Figure 10: Topology and replication metrics after 12-node convergence.

Selected Node Inspector

node-1 | status: local | direct observed by local node: yes | phi(local): n/a

Peer	Relation	Status	Phi	Evidence
node-2	direct	alive_direct	0.171	direct_heartbeat
node-3	direct	alive_direct	0.093	direct_heartbeat
node-4	direct	suspected	2.649	gossip_status
node-5	direct	alive_direct	0.298	direct_heartbeat
node-6	direct	alive_direct	0.300	gossip_status
node-7	direct	suspected	2.422	direct_heartbeat
node-8	direct	alive_direct	0.000	direct_heartbeat
node-9	direct	alive_direct	0.201	direct_heartbeat
node-10	direct	alive_direct	0.000	direct_heartbeat
node-11	direct	dead	13.934	gossip_status
node-12	direct	alive_direct	0.235	direct_heartbeat

Figure 11: Membership/liveness view (`alive`, `suspected`, `dead`).

Figure 12: Replicated sensor state with source and timestamp.

8 Self-evaluation

This project was developed individually by Alex Santini. Accordingly, this self-evaluation reflects my full personal contribution to the design, implementation, validation, deployment, and documentation of the system.

8.1 Alex Santini

Role in the project. As the sole contributor, I covered the entire development lifecycle: requirements analysis, protocol and architecture design, runtime implementation, testing, containerized deployment, observability tooling, and final report writing. This included both the distributed core and the supporting artifacts required to reproduce multi-node experiments.

Main contributions. My primary contribution was the design and implementation of a decentralized peer-to-peer sensor-state replication system based on autonomous nodes, TCP communication, JSON protocol messages, push-pull gossip, and timestamp-based Last-Writer-Wins conflict resolution. I also implemented membership and failure-observation mechanisms, including heartbeat-based liveness tracking and Phi Accrual failure suspicion, together with recovery paths based on incremental deltas and full-state synchronization.

From an operational perspective, I prepared Docker Compose topologies for reproducible multi-node simulations and configured sensor profiles, bootstrap data, and fault-related parameters through environment variables. I also developed the read-only HTTP and introspection interfaces used by tests, scripts, and the optional web dashboard to inspect sensor values, membership, topology, events, and replication metrics.

Validation was another major part of the project. I developed and maintained tests for protocol handling, networking, state convergence, membership, failure detection, sensors, web APIs, Docker-based scenarios, crash/restart recovery, and partition reconciliation. These tests helped map intended distributed-systems properties to observable behavior in repeatable experiments.

Product strengths. The strongest aspect of the project is that its core distributed mechanisms are implemented explicitly rather than delegated to a broker or external middleware. This makes gossip dissemination, membership propagation, failure suspicion, timestamp-based conflict resolution, and recovery behavior directly observable. The project also benefits from a reproducible deployment model, a clear separation between control plane, data plane, and observability plane, and a validation strategy combining unit tests, integration tests, Docker scenarios, and manual observation.

Weaknesses and limitations. The system remains a course-project prototype rather than a production-ready smart-city platform. It does not provide authentication, authorization, or transport encryption; runtime state is stored in memory rather than durable local storage; and the Last-Writer-Wins policy, while deterministic, is semantically simple. Larger topologies would also require a more systematic quantitative evaluation of gossip overhead, convergence latency, and failure-detector sensitivity.

Personal evaluation of contribution. The most successful decisions were keeping

the mandatory decentralized architecture understandable through a layered structure between the networking layer, protocol layer, and semantic modules, treating observability as a first-class concern, and validating recovery and convergence through reproducible Docker scenarios. The aspects that required the most iteration were balancing incremental anti-entropy with full synchronization and tuning failure detection under delayed or lossy communication. With more time, I would strengthen the quantitative evaluation, extend the system to more complex topology policies, add durable snapshots, and harden security boundaries while preserving the educational transparency of the implementation.

Group members.

- Alex Santini – `alex.santini2@studio.unibo.it`

9 Future works

Although the project achieves its educational objectives in terms of distributed replication, eventual convergence, and observability, the implemented architecture leaves room for several significant extensions, both from an engineering and an experimental perspective.

9.1 Security and operational hardening

The first area for evolution concerns security. In the current version, the system operates in a controlled context and does not implement peer authentication, HTTP API protection, or channel encryption. A natural evolution would be to introduce:

- mutual authentication among nodes;
- protection of HTTP endpoints through tokens or mutual TLS;
- stricter validation of the sender's logical identity with respect to the transport channel;
- authorization policies to distinguish observational and administrative roles.

These changes would bring the system closer to a real-world scenario, where fault tolerance must coexist with baseline trust requirements and control-plane protection.

9.2 Persistence and historical data

At present, nodes retain only in-memory volatile state. This choice is consistent with the project's focus, but limits the system's ability to survive extended restarts or provide long-term temporal analysis. A relevant extension would therefore be the introduction of:

- local persistence of sensor records and recent deltas;
- periodic snapshots of membership and topology metadata;
- startup restore mechanisms to reduce bootstrap cost;
- separate historical archiving for offline analysis and comparison across experimental runs.

This evolution would also enable more rigorous comparisons of *cold-start* behavior, recovery times, and temporary information loss after extended crashes.

9.3 Richer replication semantics

The adopted Last-Writer-Wins criterion is simple, deterministic, and suitable for the project context, but it is also a simplification. Looking ahead, it would be interesting to evaluate:

- the use of data-type-specific CRDTs;
- differentiated merge policies by sensor category;
- version vectors or causal metadata to better distinguish true concurrency from simple delayed arrival;
- adaptive strategies between incremental replication and full sync based on observed divergence.

These extensions would enable deeper study of the trade-off between implementation simplicity, amount of exchanged metadata, and semantic quality of convergence.

9.4 Experiments on larger topologies

Another natural direction concerns extending experimental scale. Current topologies already allow observation of interesting phenomena, but extending to larger clusters, more heterogeneous connectivity, or additional topology policies such as low-degree connection strategies would make it possible to:

- evaluate the impact of topology density on gossip traffic;
- compare full-mesh behavior with lower-degree topology policies;
- study failure-detector sensitivity in less regular networks;
- compare fan-out, sync intervals, and delta-buffer sizes under higher loads;
- verify readability and sustainability of observability as node count grows.

From an experimental standpoint, it would also be useful to complement this with systematic quantitative metrics on convergence latency, traffic volume, and recovery costs.

9.5 Advanced observability and analysis tooling

The project already provides a solid read-only introspection baseline, but it could be further extended through:

- export of metrics to standard monitoring stacks;
- automatic alerting on critical stale-data or `DELTA_UNAVAILABLE` thresholds;
- persistent temporal tracing of control-plane events;

- automated comparison across different replication and failure-detection configurations.

In this way, the system would become not only a functional prototype, but also a more mature experimental platform for systematically analyzing the consequences of architectural choices.